

Report S11/L2

Uso di Wireshark per l'Osservazione dell' Handshake a 3 Vie TCP

1. Obiettivo e Contesto

In questa esercitazione ho utilizzato Wireshark e tcpdump per catturare ed esaminare i pacchetti generati tra il browser web in esecuzione sull'host H1 e un server web HTTP in esecuzione sull'host H4 (172.16.0.40), all'interno della topologia Mininet fornita dall'ambiente CyberOps Workstation. L'obiettivo principale è stato osservare nel dettaglio l'handshake a tre vie (3-way handshake) che TCP utilizza per stabilire una sessione affidabile tra due host prima dello scambio dei dati applicativi.

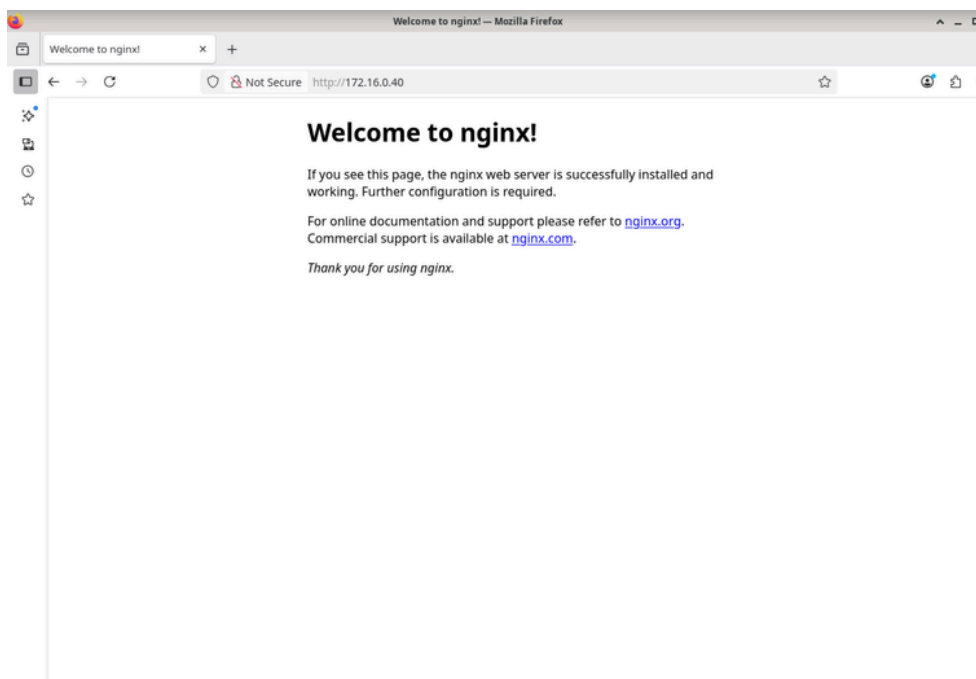
Il laboratorio è stato suddiviso in tre parti: preparazione degli host per la cattura del traffico, analisi dei pacchetti tramite l'interfaccia grafica di Wireshark e, infine, visualizzazione dei pacchetti da riga di comando tramite tcpdump.

2. Parte 1 – Preparazione degli Host per la Cattura del Traffico

Dopo aver avviato la VM CyberOps e Mininet con lo script cyberops_topo.py, ho aperto i terminali xterm degli host H1 e H4 dalla CLI di Mininet. Su H4 ho avviato il server web tramite lo script reg_server_start.sh.

2.1 Warning di nginx all'avvio del server web

All'avvio dello script ho ricevuto il seguente messaggio sul terminale Node: H4:





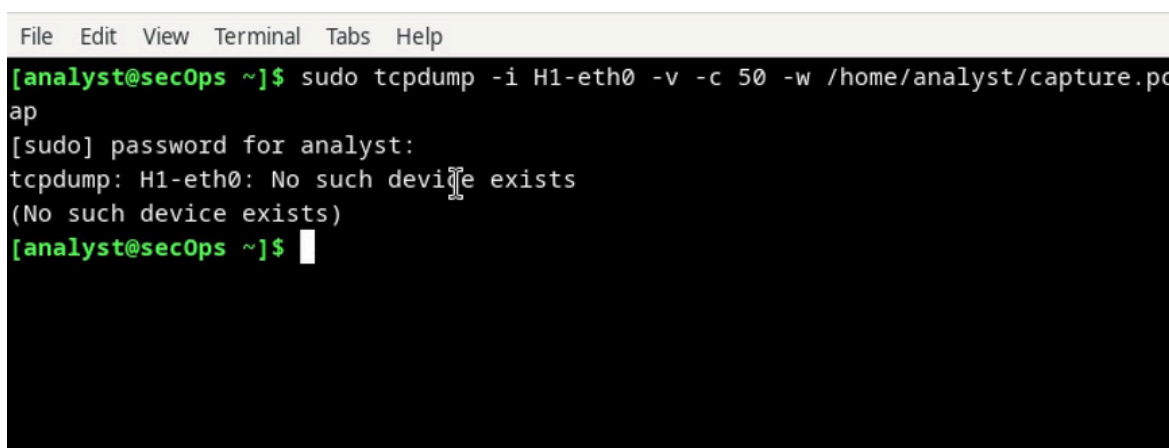
```
[root@sec0ps analyst]# /home/analyst/lab.support.files/
attack_scripts/  malware/      openssl_lab/    pox/
instructor/      mininet_services pcaps/          scripts/
[root@sec0ps analyst]# /home/analyst/lab.support.files/scripts/reg_server_start
.sh
2026/06/16 13:44:39 [warn] 1640#1640: could not build optimal types_hash, you sh
ould increase either types_hash_max_size: 1024 or types_hash_bucket_size: 64; ig
noring types_hash_bucket_size
[root@sec0ps analyst]#
```

Fig. 1 – Warning di nginx al lancio di reg_server_start.sh su H4.

Ho verificato che si trattasse di un semplice warning e non di un errore bloccante: nginx segnala che la tabella hash interna usata per il mapping dei MIME type (types_hash_max_size/types_hash_bucket_size) non è dimensionata in modo ottimale, ma il processo continua comunque ad avviarsi correttamente, come confermato dal ritorno immediato al prompt dei comandi senza alcun messaggio di tipo [emerg] o [error]. Il server web su H4 risultava quindi raggiungibile e pronto a rispondere alle richieste, come verificato successivamente raggiungendo la pagina di default "Welcome to nginx" dal browser su H1.

2.2 Errore di interfaccia durante l'avvio di tcpdump

Nel tentativo di avviare la cattura con tcpdump sull'interfaccia H1-eth0, ho inizialmente ottenuto il seguente errore:



```
File Edit View Terminal Tabs Help
[analyst@sec0ps ~]$ sudo tcpdump -i H1-eth0 -v -c 50 -w /home/analyst/capture.pcap
[sudo] password for analyst:
tcpdump: H1-eth0: No such device exists
(No such device exists)
[analyst@sec0ps ~]$
```

Fig. 2 – Errore "No such device exists" durante l'avvio di tcpdump.

L'errore era dovuto al fatto che il comando era stato lanciato nel terminale principale della VM e non all'interno della finestra xterm associata al namespace di rete dell'host H1 creato da Mininet: l'interfaccia H1-eth0 esiste infatti solo all'interno di tale namespace. Ho risolto il problema riaprendo la finestra xterm di H1 dalla CLI di Mininet (xterm H1) ed eseguendo lì il comando su analyst, l'avvio di Firefox e

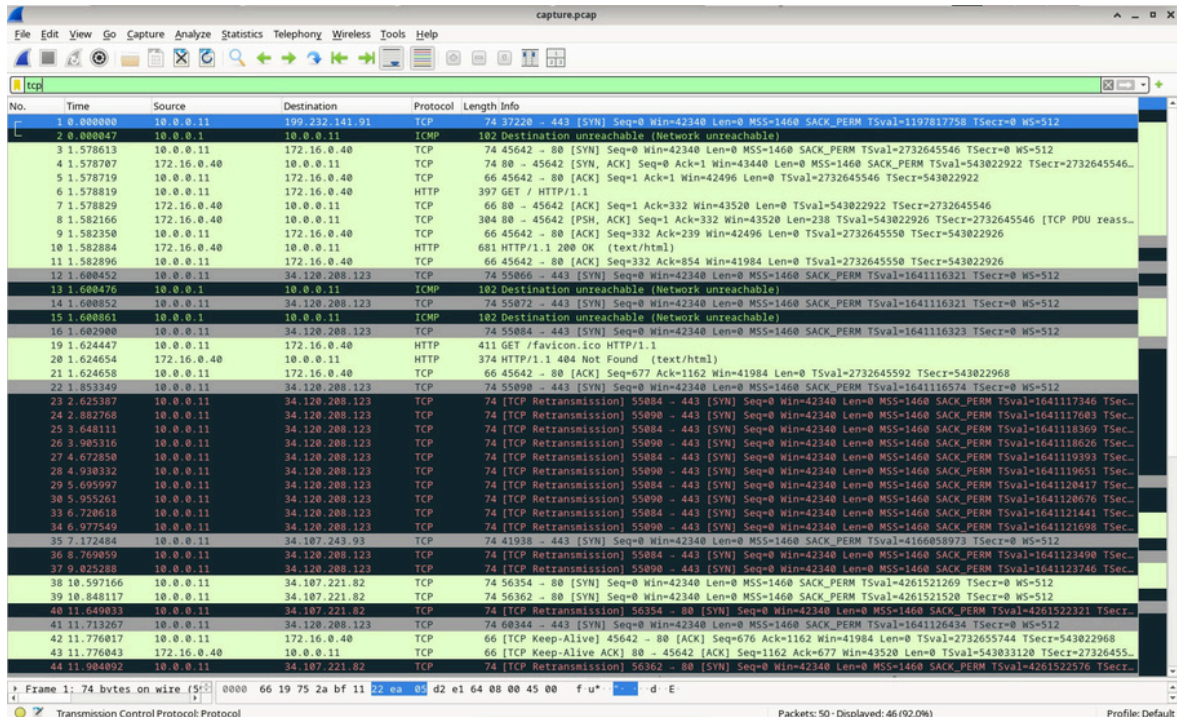
successivamente il comando tcpdump corretto:

```
sudo tcpdump -i H1-eth0 -v -c 50 -w /home/analyst/capture.pcap
```

3. Parte 2 – Analisi dei Pacchetti con Wireshark

3.1 Apertura della cattura e applicazione del filtro tcp

Ho aperto Wireshark con wireshark-gtk & sul terminale Node: H1 e caricato il file /home/analyst/capture.pcap tramite File > Open. Applicando il filtro tcp ho notato, oltre al traffico verso il server web di laboratorio, anche del traffico di sottofondo generato automaticamente da Firefox verso indirizzi esterni non instradabili dalla rete Mininet (con conseguenti risposte ICMP "Network unreachable" dal router R1) e diversi tentativi SYN ritrasmessi verso altri host esterni.

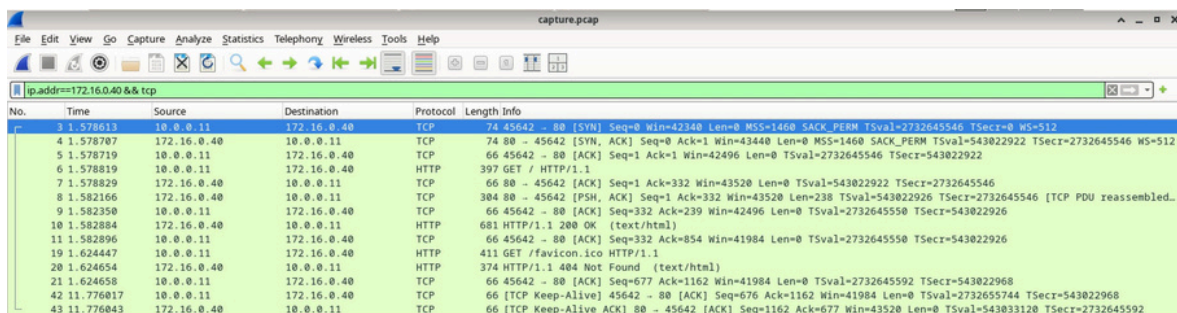


No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.0.11	192.232.141.91	TCP	74	27220 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=1197817758 TSecr=0 WS=512
2	0.000047	10.0.0.11	10.0.0.11	ICMP	182	Destination unreachable (Network unreachable)
3	1.578613	10.0.0.11	172.16.0.40	TCP	74	45642 -> 80 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=2732645546 TSecr=0 WS=512
4	1.578787	172.16.0.40	10.0.0.11	TCP	74	80 -> 45642 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM TSval=543022922 TSecr=2732645546
5	1.578719	10.0.0.11	172.16.0.40	TCP	66	45642 -> 80 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=2732645546 TSecr=543022922
6	1.578819	10.0.0.11	172.16.0.40	HTTP	397	GET / HTTP/1.1
7	1.578829	172.16.0.40	10.0.0.11	TCP	66	80 -> 45642 [ACK] Seq=1 Ack=332 Win=43520 Len=0 TSval=543022922 TSecr=2732645546
8	1.582166	172.16.0.40	10.0.0.11	TCP	384	80 -> 45642 [PSH, ACK] Seq=1 Ack=332 Win=43520 Len=238 TSval=543022926 TSecr=2732645546 [TCP PDU reassl.]
9	1.582350	10.0.0.11	172.16.0.40	TCP	66	45642 -> 80 [ACK] Seq=332 Ack=239 Win=42496 Len=0 TSval=2732645550 TSecr=543022926
10	1.582884	172.16.0.40	10.0.0.11	HTTP	681	HTTP/1.1 200 OK (text/html)
11	1.582896	10.0.0.11	172.16.0.40	TCP	66	45642 -> 80 [ACK] Seq=332 Ack=854 Win=41984 Len=0 TSval=2732645550 TSecr=543022926
12	1.600452	10.0.0.11	34.120.208.123	TCP	74	55066 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164116321 TSecr=0 WS=512
13	1.600476	10.0.0.11	10.0.0.11	ICMP	182	Destination unreachable (Network unreachable)
14	1.600552	10.0.0.11	34.120.208.123	TCP	74	55072 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164116321 TSecr=0 WS=512
15	1.600601	10.0.0.11	10.0.0.11	ICMP	182	Destination unreachable (Network unreachable)
16	1.600990	10.0.0.11	34.120.208.123	TCP	74	55084 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164116323 TSecr=0 WS=512
19	1.624447	10.0.0.11	172.16.0.40	HTTP	411	GET /favicon.ico HTTP/1.1
20	1.624654	172.16.0.40	10.0.0.11	HTTP	374	HTTP/1.1 404 Not Found (text/html)
21	1.624658	10.0.0.11	172.16.0.40	TCP	66	45642 -> 80 [ACK] Seq=677 Ack=1162 Win=41984 Len=0 TSval=2732645592 TSecr=543022968
22	1.853349	10.0.0.11	34.120.208.123	TCP	74	55090 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164116574 TSecr=0 WS=512
23	2.625387	10.0.0.11	34.120.208.123	TCP	74	[TCP Retransmission] 55084 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164117346 TSecr=0 WS=512
24	2.882768	10.0.0.11	34.120.208.123	TCP	74	[TCP Retransmission] 55090 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164117603 TSecr=0 WS=512
25	3.648111	10.0.0.11	34.120.208.123	TCP	74	[TCP Retransmission] 55084 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164118360 TSecr=0 WS=512
26	3.905316	10.0.0.11	34.120.208.123	TCP	74	[TCP Retransmission] 55090 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164118626 TSecr=0 WS=512
27	4.672850	10.0.0.11	34.120.208.123	TCP	74	[TCP Retransmission] 55084 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164119393 TSecr=0 WS=512
28	4.938332	10.0.0.11	34.120.208.123	TCP	74	[TCP Retransmission] 55090 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164119651 TSecr=0 WS=512
29	5.695997	10.0.0.11	34.120.208.123	TCP	74	[TCP Retransmission] 55084 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164120417 TSecr=0 WS=512
30	5.955261	10.0.0.11	34.120.208.123	TCP	74	[TCP Retransmission] 55090 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164120676 TSecr=0 WS=512
33	6.728618	10.0.0.11	34.120.208.123	TCP	74	[TCP Retransmission] 55084 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164121441 TSecr=0 WS=512
34	6.977549	10.0.0.11	34.120.208.123	TCP	74	[TCP Retransmission] 55090 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164121698 TSecr=0 WS=512
35	7.372454	10.0.0.11	34.120.208.123	TCP	74	[TCP Retransmission] 55084 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164122464 TSecr=0 WS=512
36	8.769059	10.0.0.11	34.120.208.123	TCP	74	[TCP Retransmission] 55084 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164123490 TSecr=0 WS=512
37	9.025288	10.0.0.11	34.120.208.123	TCP	74	[TCP Retransmission] 55090 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164123746 TSecr=0 WS=512
38	10.597166	10.0.0.11	34.107.221.82	TCP	74	56354 -> 80 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=4261521269 TSecr=0 WS=512
39	10.848117	10.0.0.11	34.107.221.82	TCP	74	56362 -> 80 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=4261521520 TSecr=0 WS=512
40	11.649833	10.0.0.11	34.107.221.82	TCP	74	[TCP Retransmission] 56354 -> 80 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=4261522321 TSecr=0 WS=512
41	11.713267	10.0.0.11	34.120.208.123	TCP	74	68344 -> 443 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=164126434 TSecr=0 WS=512
42	11.776817	10.0.0.11	172.16.0.40	TCP	66	[TCP Keep-Alive] 45642 -> 80 [ACK] Seq=676 Ack=1162 Win=41984 Len=0 TSval=2732655744 TSecr=543022968
43	11.776843	172.16.0.40	10.0.0.11	TCP	66	[TCP Keep-Alive] 80 -> 45642 [ACK] Seq=1162 Ack=677 Win=43520 Len=0 TSval=543033120 TSecr=2732645592
44	11.984092	10.0.0.11	34.107.221.82	TCP	74	[TCP Retransmission] 56362 -> 80 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=426152576 TSecr=0 WS=512

Fig. 3 – Cattura completa con filtro "tcp": traffico verso 172.16.0.40 misto a traffico di sottofondo del browser.

Per isolare unicamente il traffico rilevante per il laboratorio, ho affinato il filtro come segue:

```
ip.addr==172.16.0.40 && tcp
```



No.	Time	Source	Destination	Protocol	Length	Info
3	1.578613	10.0.0.11	172.16.0.40	TCP	74	45642 -> 80 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=2732645546 TSecr=0 WS=512
4	1.578787	172.16.0.40	10.0.0.11	TCP	74	80 -> 45642 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM TSval=543022922 TSecr=2732645546
5	1.578719	10.0.0.11	172.16.0.40	TCP	66	45642 -> 80 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=2732645546 TSecr=543022922
6	1.578819	10.0.0.11	172.16.0.40	HTTP	397	GET / HTTP/1.1
7	1.578829	172.16.0.40	10.0.0.11	TCP	66	80 -> 45642 [ACK] Seq=1 Ack=332 Win=43520 Len=0 TSval=543022922 TSecr=2732645546
8	1.582166	172.16.0.40	10.0.0.11	TCP	384	80 -> 45642 [PSH, ACK] Seq=1 Ack=332 Win=43520 Len=238 TSval=543022926 TSecr=2732645546 [TCP PDU reassmbled.]
9	1.582350	10.0.0.11	172.16.0.40	TCP	66	45642 -> 80 [ACK] Seq=332 Ack=239 Win=42496 Len=0 TSval=2732645550 TSecr=543022926
10	1.582884	172.16.0.40	10.0.0.11	HTTP	681	HTTP/1.1 200 OK (text/html)
11	1.582896	10.0.0.11	172.16.0.40	TCP	66	45642 -> 80 [ACK] Seq=332 Ack=854 Win=41984 Len=0 TSval=2732645550 TSecr=543022926
19	1.624447	10.0.0.11	172.16.0.40	HTTP	411	GET /favicon.ico HTTP/1.1
20	1.624654	172.16.0.40	10.0.0.11	HTTP	374	HTTP/1.1 404 Not Found (text/html)
21	1.624658	10.0.0.11	172.16.0.40	TCP	66	45642 -> 80 [ACK] Seq=677 Ack=1162 Win=41984 Len=0 TSval=2732645592 TSecr=543022968
42	11.776817	10.0.0.11	172.16.0.40	TCP	66	[TCP Keep-Alive] 45642 -> 80 [ACK] Seq=676 Ack=1162 Win=41984 Len=0 TSval=2732655744 TSecr=543022968
43	11.776843	172.16.0.40	10.0.0.11	TCP	66	[TCP Keep-Alive] 80 -> 45642 [ACK] Seq=1162 Ack=677 Win=43520 Len=0 TSval=543033120 TSecr=2732645592

Fig. 4 – Traffico filtrato verso/da il server web 172.16.0.40: handshake a 3 vie (frame 3-4-5) seguito dalla sessione HTTP.

Con questo filtro, l'handshake a tre vie corrisponde ai frame 3, 4 e 5 della cattura (i frame 1 e 2 appartengono al traffico di sottofondo scartato). Si osserva inoltre il resto della sessione applicativa: la richiesta HTTP GET /, la risposta 200 OK, una successiva richiesta GET /favicon.ico con risposta 404 Not Found (nginx non fornisce una favicon di default) e infine un keep-alive.

3.2 Analisi dettagliata dei pacchetti dell'handshake

Frame 3 – Primo pacchetto (SYN)

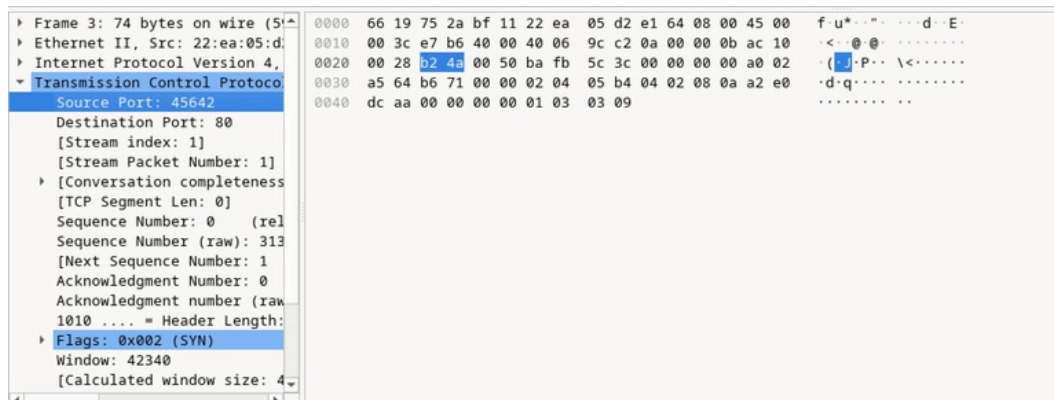


Fig. 5 – Dettaglio del frame 3: pacchetto SYN inviato da H1 al server web.

Qual è il numero di porta TCP di origine?	45642
Come classificherei la porta di origine?	Porta dinamica/effimera (privata), assegnata automaticamente dal sistema operativo client per la sessione.
Qual è il numero di porta TCP di destinazione?	80
Come classificherei la porta di destinazione?	Porta well-known, standard per il protocollo HTTP.
Quale flag è impostato?	SYN (0x002)
A quale valore è impostato il numero di sequenza relativo?	0

Frame 4 – Secondo pacchetto (SYN, ACK)

Il secondo pacchetto rappresenta la risposta del server web alla richiesta iniziale di avviare una sessione.

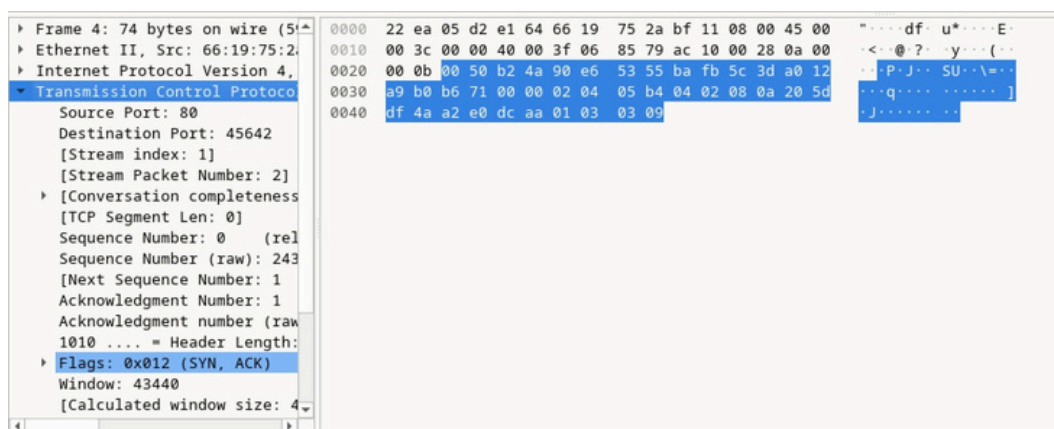


Fig. 6 – Dettaglio del frame 4: risposta SYN, ACK del server web.

Quali sono i valori delle porte di origine e destinazione?	Porta di origine: 80 (server web). Porta di destinazione: 45642 (client H1).
Quali flag sono impostati?	SYN, ACK (0x012)
A quali valori sono impostati i numeri relativi di sequenza e acknowledgment?	Sequence number relativo: 0. Acknowledgment number relativo: 1 (conferma di ricezione del SYN di H1, seq+1).

Frame 5 – Terzo pacchetto (ACK)

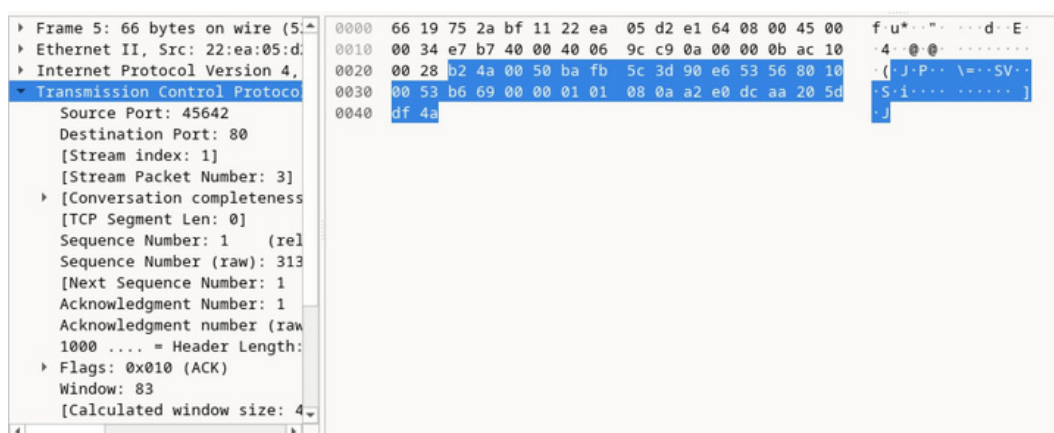


Fig. 7 – Dettaglio del frame 5: pacchetto ACK finale che completa l'handshake.

Quale flag è impostato?	ACK (0x010)
Valori dei numeri relativi di sequenza e acknowledgment	Sequence number relativo: 1. Acknowledgment number relativo: 1. Entrambi i lati hanno confermato i numeri di sequenza dell'altro, partendo dal valore 1 come punto di riferimento relativo.

A questo punto la connessione TCP tra H1 e il server web su H4 risulta stabilita e la comunicazione applicativa (HTTP) può avere inizio, come confermato dai frame successivi osservati nella cattura (GET / e risposta 200 OK).

4. Parte 3 – Visualizzazione dei Pacchetti con tcpdump

Oltre all'analisi grafica con Wireshark, la traccia di laboratorio prevede la visualizzazione del medesimo file di cattura da riga di comando, consultando le pagine di manuale di tcpdump (man tcpdump) per individuare le opzioni utili a filtrare le informazioni desiderate dal file pcap.

Cosa fa l'opzione -r?

L'opzione -r <file> indica a tcpdump di leggere i pacchetti da un file di cattura salvato su disco (ad esempio un file .pcap), invece di catturarli dal vivo da un'interfaccia di rete. È l'opzione complementare a -w, usata invece per scrivere i pacchetti catturati su file.

Applicando il comando indicato nella traccia sul file generato in precedenza:

```
tcpdump -r /home/analyst/capture.pcap tcp -c 3
```

è possibile visualizzare gli stessi primi pacchetti TCP già analizzati in Wireshark (SYN, SYN/ACK, ACK), confermando da riga di comando la sequenza dell'handshake osservata graficamente nella Parte 2. Per visualizzare l'handshake completo nel caso in cui il traffico di sottofondo precedesse i pacchetti di interesse, è sufficiente aumentare il valore passato a -c.

Al termine dell'analisi, ho chiuso Mininet con il comando quit dalla CLI principale e ho eseguito sudo mn -c per ripulire i processi e le interfacce di rete virtuali create durante la sessione di laboratorio.

5. Domande di Riflessione

1. Ci sono centinaia di filtri disponibili in Wireshark. Una rete di grandi dimensioni potrebbe avere numerosi filtri e molti tipi diversi di traffico. Elenca tre filtri che potrebbero essere utili a un amministratore di rete.

Tra i numerosi filtri disponibili, ne ritengo particolarmente utili tre per un amministratore di rete. Il filtro `tcp.flags.syn==1 && tcp.flags.ack==0` permette di isolare i soli pacchetti SYN "puri", utili per individuare anomalie come scansioni di porte (port scanning) o tentativi di SYN flood. Il filtro `tcp.analysis.retransmission` consente di individuare rapidamente le ritrasmissioni TCP, spesso sintomo di congestione di rete, perdita di pacchetti o problemi su un particolare segmento. Infine, il filtro `dns` permette di isolare tutte le richieste e risposte DNS, utile sia per il troubleshooting della risoluzione dei nomi sia per individuare comportamenti sospetti come query verso domini generati algebricamente (DGA), spesso associati a malware.

2. In quali altri modi Wireshark potrebbe essere utilizzato in una rete di produzione?

In un ambiente di produzione, oltre alla didattica sui protocolli, Wireshark può essere impiegato per il troubleshooting delle prestazioni di rete, ad esempio per individuare latenze anomale, ritrasmissioni eccessive o problemi di window scaling tra applicazioni e server. È utile anche in ambito forense, per analizzare il traffico catturato durante o dopo un incidente di sicurezza e ricostruire le azioni di un eventuale attaccante (ad esempio comunicazioni di Command and Control o esfiltrazione di dati). Può inoltre essere utilizzato per stabilire una baseline del traffico normale di una rete, facilitando l'individuazione di pattern anomali nel tempo, oppure per verificare la corretta configurazione di protocolli di sicurezza, come il controllo dei parametri negoziati durante un handshake TLS o la verifica della corretta cifratura del traffico applicativo.

6. Conclusioni

Questo laboratorio mi ha permesso di osservare concretamente il funzionamento dell'handshake a tre vie TCP, dalla generazione del traffico tramite browser fino all'analisi dettagliata dei singoli pacchetti con Wireshark e tcpdump. Ho inoltre affrontato e risolto due piccoli problemi operativi (il warning di nginx in fase di avvio del server web e l'errore di interfaccia di rete dovuto all'esecuzione del comando nel terminale errato), che mi hanno aiutato a comprendere meglio sia il funzionamento interno di nginx sia la gestione dei namespace di rete virtuali creati da Mininet.